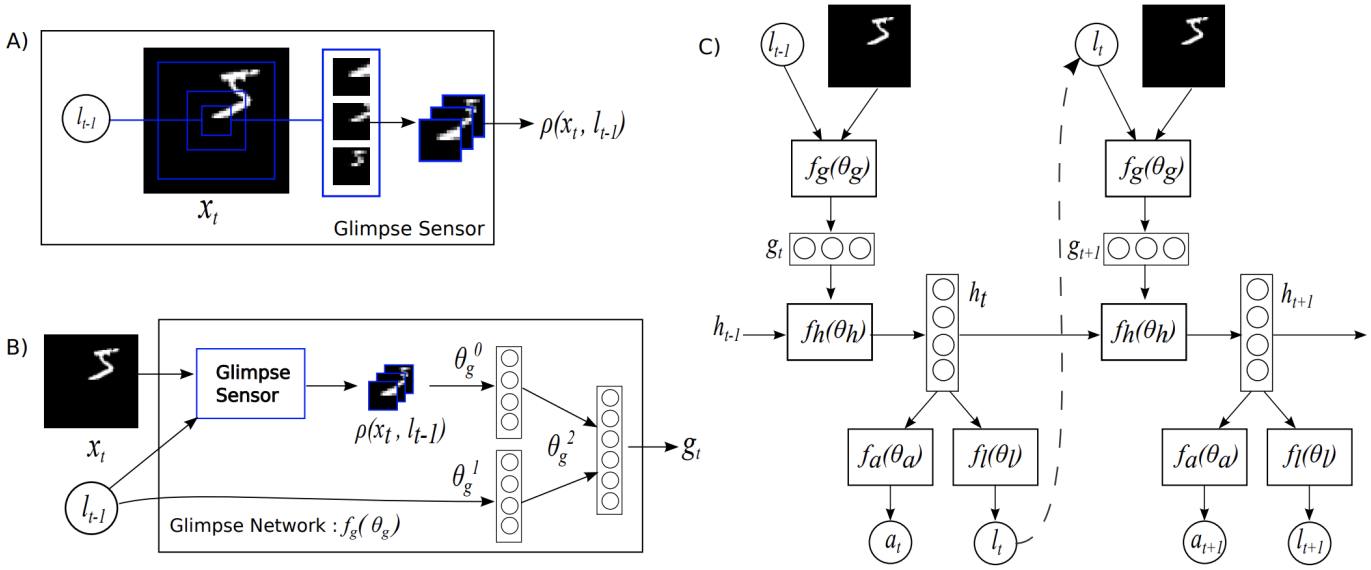

Fig. 6. Attention process in RAM [31]. (A): a glimpse sensor takes image and center coordinates as input and outputs multiple resolution patches. (B): a glimpse network includes a glimpse sensor, taking image and center coordinates as input and outputting a feature vector. (C) the entire network recurrently uses a glimpse network, outputting the predicted result as well as the next center coordinates. Figure is taken from [31].

coordinates in the output feature map, while x_i^t and y_i^t are corresponding coordinates in the input feature map and the θ matrix is the learnable affine matrix. After obtaining the correspondence, the network can sample relevant input regions using the correspondence. To ensure that the whole process is differentiable and can be updated in an end-to-end manner, bilinear sampling is used to sample the input features

STNs focus on discriminative regions automatically and learn invariance to some geometric transformations.

3.3.6 Deformable Convolutional Networks

With similar purpose to STNs, Dai et al. [7] proposed *deformable convolutional networks* (deformable ConvNets) to be invariant to geometric transformations, but they pay attention to the important regions in a different manner.

Specifically, deformable ConvNets do not learn an affine transformation. They divide convolution into two steps, firstly sampling features on a regular grid $\mathcal{R}$ from the input feature map, then aggregating sampled features by weighted summation using a convolution kernel. The process can be written as:

$$Y(p_0) = \sum_{p_i \in \mathcal{R}} w(p_i) X(p_0 + p_i) \tag{43}$$

$$\mathcal{R} = \{(-1,-1),(-1,0),\ldots,(1,1)\} \tag{44}$$

The deformable convolution augments the sampling process by introducing a group of learnable offsets Δp_i which can be generated by a lightweight CNN. Using the offsets Δp_i, the deformable convolution can be formulated as:

$$Y(p_0) = \sum_{p_i \in \mathcal{R}} w(p_i) X(p_0 + p_i + \Delta p_i). \tag{45}$$

Through the above method, adaptive sampling is achieved. However, Δp_i is a floating point value unsuited to grid sampling. To address this problem, bilinear interpolation is used. Deformable RoI pooling is also used, which greatly improves object detection.

Deformable ConvNets adaptively select the important regions and enlarge the valid receptive field of convolutional neural networks; this is important in object detection and semantic segmentation tasks.

3.3.7 Self-attention and variants

Self-attention was proposed and has had great success in the field of *natural language processing* (NLP) [33], [38], [39], [149], [150], [151], [152]. Recently, it has also shown the potential to become a dominant tool in computer vision [8], [15], [34], [78], [153]. Typically, self-attention is used as a spatial attention mechanism to capture global information. We now summarize the self-attention mechanism and its common variants in computer vision.

Due to the localisation of the convolutional operation, CNNs have inherently narrow receptive fields [154], [155], which limits the ability of CNNs to understand scenes globally. To increase the receptive field, Wang et al. [15] introduced self-attention into computer vision.

Taking a 2D image as an example, given a feature map $F \in \mathbb{R}^{C \times H \times W}$, self-attention first computes the queries, keys and values $Q, K, V \in \mathbb{R}^{C' \times N}, N = H \times W$ by linear projection and reshaping operations. Then self-attention can be formulated as:

$$A = (a)_{i,j} = \text{Softmax}(QK^T), \tag{46}$$

$$Y = AV, \tag{47}$$

where $A \in \mathbb{R}^{N \times N}$ is the attention matrix and $\alpha_{i,j}$ is the relationship between the i-th and j-th elements. The whole process is shown in Fig. 7(left). Self-attention is a powerful tool to model global information and is useful in many visual tasks [9], [22], [26], [62], [63], [64], [65], [66], [67].

However, the self-attention mechanism has several shortcomings, particularly its quadratic complexity, which limit its

applicability. Several variants have been introduced to alleviate these problems. The *disentangled non-local* approach [74] improves self-attention's accuracy and effectiveness, but most variants focus on reducing its computational complexity.

CCNet [41] regards the self-attention operation as a graph convolution and replaces the densely-connected graph processed by self-attention with several sparsely-connected graphs. To do so, it proposes *criss-cross attention* which considers row attention and column attention recurrently to obtain global information. CCNet reduces the complexity of self-attention from $O(N^2)$ to $O(N\sqrt{N})$.

EMANet [40] views self-attention in terms of expectation maximization (EM). It proposes *EM attention* which adopts the EM algorithm to get a set of compact bases instead of using all points as reconstruction bases. This reduces the complexity from $O(N^2)$ to $O(NK)$, where K is the number of compact bases.

ANN [68] suggests that using all positional features as key and vectors is redundant and adopts spatial pyramid pooling [156], [157] to obtain a few representative key and value features to use instead, to reduce computation.

GCNet [69] analyses the attention map used in self-attention and finds that the global contexts obtained by self-attention are similar for different query positions in the same image. Thus, it first proposes to predict a single attention map shared by all query points, and then gets global information from a weighted sum of input features according to this attention map. This is like average pooling, but is a more general process for collecting global information.

A^2Net [70] is motivated by SENet to divide attention into feature gathering and feature distribution processes, using two different kinds of attention. The first aggregates global information via second-order attention pooling and the second distributes the global descriptors by soft selection attention.

GloRe [71] understands self-attention from a graph learning perspective. It first collects N input features into $M \ll N$ nodes and then learns an adjacency matrix of global interactions between nodes. Finally, the nodes distribute global information to input features. A similar idea can be found in LatentGNN [72], MLP-Mixer [158] and ResMLP [159].

OCRNet [73] proposes the concept of *object-contextual representation* which is a weighted aggregation of all object regions' representations in the same category, such as a weighted average of all car region representations. It replaces the key and vector with this object-contextual representation leading to successful improvements in both speed and effectiveness.

The *disentangled non-local* approach was motivated by [15], [69]. Yin et al [74] deeply analyzed the self-attention mechanism resulting in the core idea of decoupling self-attention into a pairwise term and a unary term. The pairwise term focuses on modeling relationships while the unary term focuses on salient boundaries. This decomposition prevents unwanted interactions between the two terms, greatly improving semantic segmentation, object detection and action recognition.

HamNet [42] models capturing global relationships as a low-rank completion problem and designs a series of white-box methods to capture global context using matrix decomposition. This not only reduces the complexity, but increases the interpretability of self-attention.

EANet [75] proposes that self-attention should only consider correlation in a single sample and should ignore potential relationships between different samples. To explore the correlation between different samples and reduce computation, it makes use of an external attention that adopts learnable, lightweight and shared key and value vectors. It further reveals that using softmax to normalize the attention map is not optimal and presents double normalization as a better alternative.

In addition to being a complementary approach to CNNs, self-attention also can be used to replace convolution operations for aggregating neighborhood information. Convolution operations can be formulated as dot products between the input feature X and a convolution kernel W:

$$Y^c_{i,j} = \sum_{a,b\in\{0,\dots,k-1\}} W_{a,b,c} X_{\hat{a},\hat{b}} \tag{48}$$

where

$$\hat{a} = i + a - \lfloor k/2 \rfloor, \qquad \hat{b} = j + b - \lfloor k/2 \rfloor, \tag{49}$$

k is the kernel size and c indicates the channel. The above formulation can be viewed as a process of aggregating neighborhood information by using a weighted sum through a convolution kernel. The process of aggregating neighborhood information can be defined more generally as:

$$Y_{i,j} = \sum_{a,b\in\{0,\dots,k-1\}} \mathrm{Rel}(i,j,\hat{a},\hat{b}) f(X_{\hat{a},\hat{b}}) \tag{50}$$

where $\mathrm{Rel}(i,j,\hat{a},\hat{b})$ is the relation between position (i,j) and position ($\hat{a}$, $\hat{b}$). With this definition, local self-attention is a special case.

For example, SASA [43] writes this as

$$Y_{i,j} = \sum_{a,b\in\mathcal{N}_k(i,j)} \mathrm{Softmax}_{ab}(q^T_{ij} k_{ab} + q_{ij} r_{a-i,b-j}) v_{ab} \tag{51}$$

where q, k and v are linear projections of input feature x, and $r_{a-i,b-j}$ is the relative positional embedding of (i,j) and (a,b).

We now consider several specific works using local self-attention as basic neural network blocks

SASA [43] suggests that using self-attention to collect global information is too computationally intensive and instead adopts local self-attention to replace all spatial convolution in a CNN. The authors show that doing so improves speed, number of parameters and quality of results. They also explores the behavior of positional embedding and show that relative positional embeddings [160] are suitable. Their work also studies how to combinie local self-attention with convolution.

LR-Net [76] appeared concurrently with SASA. It also studies how to model local relationships by using local self-attention. A comprehensive study probed the effects of positional embedding, kernel size, appearance composability and adversarial attacks.

SAN [77] explored two modes, pairwise and patchwise, of utilizing attention for local feature aggregation. It proposed a novel vector attention adaptive both in content and

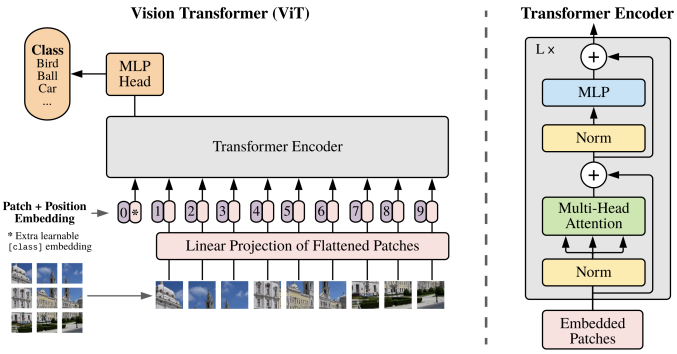

Fig. 7. Vision transformer [34]. Left: architecture. Vision transformer first splits the image into different patches and projects them into feature space where a transformer encoder processes them to produce the final result. Right: basic vision transformer block with multi-head attention core. Figure is taken from [34].

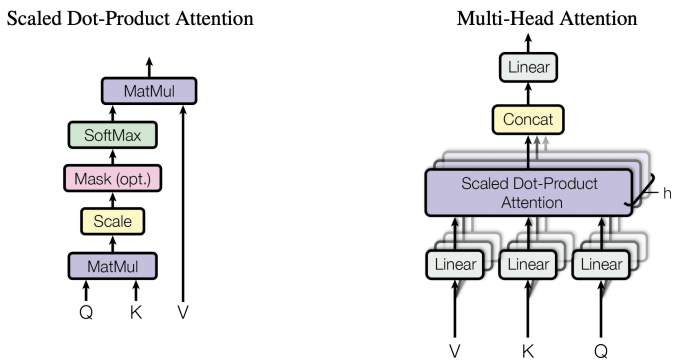

Fig. 8. Left: Self-attention. Right: Multi-head self-attention. Figure from [33].

channel, and assessed its effectiveness both theoretically and practically. In addition to providing significant improvements in the image domain, it also has been proven useful in 3D point cloud processing [80].

3.3.8 Vision Transformers

Transformers have had great success in natural language processing [33], [38], [149], [150], [152], [161]. Recently, iGPT [78] and DETR [8] demonstrated the huge potential for transformer-based models in computer vision. Motivated by this, Dosovitskiy et al [34] proposed the *vision transformer* (ViT) which is the first pure transformer architecture for image processing. It is capable of achieving comparable results to modern convolutional neural networks.

As Fig 7 shows, the main part of ViT is the multi-head attention (MHA) module. MHA takes a sequence as input. It first concatenates a class token with the input feature $F \in \mathcal{R}^{N\times C}$, where N is the number of pixels. Then it gets $Q, K \in \mathcal{R}^{N\times C'}$ and $V \in \mathcal{R}^{N\times C}$ by linear projection. Next, Q, K and V are divided into H heads in the channel domain and self-attention separately applied to them. The MHA approach is shown in Fig. 8. ViT stacks a number of MHA layers with fully connected layers, layer normalization [162] and the GELU [163] activation function.

ViT demonstrates that a pure attention-based network can achieve better results than a convolutional neural network especially for large datasets such as JFT-300 [164] and ImageNet-21K [165].

Following ViT, many transformer-based architectures such as PCT [27], IPT [79], T2T-ViT [44], DeepViT [166], SETR [81], PVT [45], CaiT [167], TNT [82], Swin-transformer [46], Query2Label [83], MoCoV3 [84], BEiT [85], SegFormer [86], FuseFormer [168] and MAE [169] have appeared, with excellent results for many kind of visual tasks including image classification, object detection, semantic segmentation, point cloud processing, action recognition and self-supervised learning.

A detailed survey of vision transformers is omitted here as other recent surveys [141], [142], [143], [170] comprehensively review the use of transformer methods for visual tasks.

3.3.9 GENet

Inspired by SENet, Hu et al. [61] designed GENet to capture long-range spatial contextual information by providing a recalibration function in the spatial domain.

GENet combines part gathering and excitation operations. In the first step, it aggregates input features over large neighborhoods and models the relationship between different spatial locations. In the second step, it first generates an attention map of the same size as the input feature map, using interpolation. Then each position in the input feature map is scaled by multiplying by the corresponding element in the attention map. This process can be described by:

$$g = f_{\text{gather}}(X), \tag{52}$$

$$s = f_{\text{excite}}(g) = \sigma(\text{Interp}(g)), \tag{53}$$

$$Y = sX. \tag{54}$$

Here, f_{gather} can take any form which captures spatial correlations, such as global average pooling or a sequence of depth-wise convolutions; $\text{Interp}(\cdot)$ denotes interpolation.

The gather-excite module is lightweight and can be inserted into each residual unit like an SE block. It emphasizes important features while suppressing noise.

3.3.10 PSANet

Motivated by success in capturing long-range dependencies in convolutional neural networks, Zhao et al. [87] presented the novel PSANet framework to aggregate global information. It models information aggregation as an information flow and proposes a bidirectional information propagation mechanism to make information flow globally.

PSANet formulates information aggregation as:

$$z_i = \sum_{j\in\Omega(i)} F(x_i, x_j, \Delta_{ij})x_j \tag{55}$$

where Δ_{ij} indicates the positional relationship between i and j. $F(x_i, x_j, \Delta_{ij})$ is a function that takes x_i, x_j and Δ_{ij} into consideration to controls information flow from j to i. Ω_i represents the aggregation neighborhood of position i; if we wish to capture global information, Ω_i should include all spatial positions.

Due to the complexity of calculating function $F(x_i, x_j, \Delta_{ij})$, it is decomposed into an approximation:

$$F(x_i, x_j, \Delta_{ij}) \approx F_{\Delta_{ij}}(x_i) + F_{\Delta_{ij}}(x_j) \tag{56}$$